

# Secure Coding Review Report

## Introduction

Secure coding practices are essential for developing reliable and safe software applications. Vulnerabilities in source code can expose applications to cyberattacks, data breaches, and unauthorized access. This report presents a security review of a Python-based login application to identify common vulnerabilities and recommend best practices for secure coding.

## Application Selected

### Python Login System

The application allows users to enter their username and password for authentication. Since login systems handle sensitive information, they must be designed securely to prevent attacks.

### Vulnerability 1: Hardcoded Passwords

Hardcoded passwords are stored directly in the source code. Anyone with access to the code can view the credentials, leading to security risks.

Example:

```
password = "admin123"
```

Recommendation

- Store passwords securely.
- Use password hashing techniques such as bcrypt.
- Avoid exposing sensitive information in source code.

### Vulnerability 2: SQL Injection

SQL Injection occurs when user input is directly included in SQL queries without validation.

Unsafe Example:

```
query = "SELECT * FROM users WHERE username='" + username + "'"
```

Attackers may manipulate the query and gain unauthorized access.

## Recommendation

- Use parameterized queries.
- Validate and sanitize user inputs.

### **Vulnerability 3: Weak Password Policy**

Weak passwords are easy to guess and increase the risk of unauthorized access.

Examples of weak passwords:

- 123456
- password
- admin

## Recommendation

Passwords should contain:

- At least 8 characters.
- Uppercase and lowercase letters.
- Numbers.
- Special characters.

### **Vulnerability 4: Lack of Input Validation**

Applications that do not validate input are vulnerable to malicious data and unexpected behavior.

## Recommendation

- Verify all user inputs.
- Restrict invalid characters.
- Apply data sanitization techniques.

### **Vulnerability 5: Poor Error Handling**

Improper error handling can expose internal information and make applications unstable.

Unsafe Example:

```
num = 10 / 0
```

Recommendation

Use exception handling:

try:

```
    num = 10 / 0
```

except ZeroDivisionError:

```
    print("Cannot divide by zero")
```

Static Analysis Tools

Several tools can be used to detect vulnerabilities and improve code quality.

| Tool      | Purpose                      |
|-----------|------------------------------|
| Bandit    | Security analysis for Python |
| Pylint    | Code quality checking        |
| SonarQube | Static code analysis         |
| Flake8    | Syntax and style analysis    |

Manual Inspection

Manual code review helps identify vulnerabilities that automated tools may miss. Developers examine source code carefully and verify whether secure coding practices are followed.

Secure Coding Best Practices

- Avoid hardcoded credentials.
- Validate and sanitize inputs.
- Use encryption and password hashing.
- Implement proper exception handling.
- Keep libraries and dependencies updated.

- Follow the principle of least privilege.
- Perform regular security testing.

## **Findings**

The secure coding review identified several security weaknesses:

1. Hardcoded passwords.
2. SQL Injection vulnerabilities.
3. Weak password policies.
4. Lack of input validation.
5. Poor exception handling.

These issues may lead to unauthorized access, information disclosure, and application failures.

## **Remediation Steps**

To improve application security, the following measures are recommended:

- Replace hardcoded passwords with hashed credentials.
- Use parameterized SQL queries.
- Enforce strong password requirements.
- Validate and sanitize user inputs.
- Implement exception handling.
- Perform regular code reviews and vulnerability assessments.

## **Benefits of Secure Coding**

Secure coding practices provide several advantages:

- Protection against cyberattacks.
- Improved software reliability.
- Reduced risk of data breaches.
- Better compliance with security standards.

## Conclusion

Secure coding is a critical aspect of software development. The review of the Python login application revealed multiple vulnerabilities that could compromise security. By implementing recommended practices such as input validation, parameterized queries, password hashing, and regular code reviews, developers can significantly enhance application security and reduce cyber risks. Continuous monitoring and adherence to secure coding principles are essential for building robust and reliable software systems.

## References

1. OWASP Top 10 Security Risks.
2. Python Documentation.
3. Bandit Security Analyzer Documentation.
4. SonarQube Documentation.
5. Flake8 Documentation.
6. Pylint User Guide.